

BUILD FULLSTACK MOBILE USING AI

COMPLETE AI PROMPT



Masynctech

Comprehensive prompt template specifically for React Native with Expo and Supabase projects:

I want to create a new React Native mobile app called [PROJECT_NAME] using Expo and Supabase. Help me set up the complete project structure and get started with the following requirements:

Project Overview:

- App Name: [PROJECT_NAME]
- Purpose: [Brief description of what the app does]
- Target Users: [Who will use this app]
- Main Features: [List 3-5 core features]

Technical Stack:

- **Frontend:** React Native with Expo (latest stable version)
- **Backend:** Supabase (database, authentication, real-time features)
- **TypeScript:** Yes (strongly typed)
- **Navigation:** React Navigation v6
- File Based Routing

State Management:

- Context API
- Redux Toolkit
- Zustand or "recommend best option"

UI Components:

- NativeBase
- Tamagui
- Custom or "recommend best option"

Authentication Requirements:

- Auth Methods: [Email/Password, Social Login (Google, Apple), Phone, etc.]
- User Profiles: [Yes/No - if yes, specify what fields are needed]
- Onboarding Flow: [Yes/No - if yes, specify number of steps]

Database & Features:

- Core Tables: [List main data entities - e.g., users, posts, comments, etc.]
- Real-time Features: [Yes/No - if yes, specify what updates in real-time]
- File Storage: [Yes/No - if yes, specify what types of files]
- Push Notifications: [Yes/No - if yes, specify notification types]

Project Setup:

1. Initialize Expo project with TypeScript configuration
2. Set up Supabase with complete database schema, RLS policies, and types
3. Configure authentication with chosen methods and user management
4. Create navigation structure with tab/stack navigation as needed
5. Set up UI components with consistent theming and styling
6. Implement core features with CRUD operations and real-time updates
7. Add proper error handling and loading states
8. Include environment configuration for development/production

Output Requirements:

- Create complete Expo project structure ready to run
- Generate `supabase/database-schema.sql` with all tables and policies
- Set up `lib/supabase.ts` with proper configuration
- Create authentication context and screens (login, signup, onboarding)
- Build main app screens with navigation
- Include TypeScript types for all data structures
- Add proper folder structure: `components/`, `screens/`, `hooks/`, `constants/`, etc.
- Create comprehensive `README.md` with setup instructions
- Include `app.json` with proper configuration

The project should include:

- ☒ Modern React Native architecture with hooks
- ☒ Expo SDK latest features and best practices
- ☒ Complete Supabase integration (auth, database, real-time)
- ☒ TypeScript throughout with proper types
- ☒ Responsive design for different screen sizes
- ☒ Production-ready security (RLS policies, input validation)
- ☒ Error boundaries and proper error handling
- ☒ Loading states and smooth user experience
- ☒ Scalable folder structure and code organization

Additional Features: [Any specific requirements like offline support, camera integration, location services, etc.]

Design Preferences:[Any specific design system, color schemes, or UI preferences]"

Example Usage:

I want to create a new React Native mobile app called FoodTracker using Expo and Supabase. Help me set up the complete project structure and get started with the following requirements:

Project Overview:

- App Name: FoodTracker
- Purpose: Personal nutrition tracking with meal logging and progress analytics
- Target Users: Health-conscious individuals wanting to track their daily nutrition
- Main Features: Meal logging, nutrition analytics, progress tracking, goal setting, photo capture

Technical Stack:

- Frontend: React Native with Expo (latest stable version)
- Backend: Supabase (database, authentication, real-time features)
- TypeScript: Yes (strongly typed)
- Navigation: React Navigation v6
- State Management: Context API
- UI Components: NativeBase

Authentication Requirements:

- Auth Methods: Email/Password, Google Sign-in

- User Profiles: Yes (name, age, weight, height, goals, preferences)
- Onboarding Flow: Yes (3 steps - personal info, goals, preferences)

Database & Features:

- Core Tables: users, meals, foods, nutrition_data, goals, progress
- Real-time Features: No
- File Storage: Yes (meal photos)
- Push Notifications: Yes (meal reminders, goal achievements)

Additional Features: Camera integration for meal photos, barcode scanning for food items, offline meal logging

Design Preferences: Modern, clean design with green/blue color scheme, focus on data visualization

Quick Customization:

1. Replace these key placeholders:

- [PROJECT_NAME] → Your app name
- [Brief description] → What your app does
- [List 3-5 core features] → Main functionality
- [Core Tables] → Your main data entities

2. Choose your preferences:

- State management solution
- UI component library
- Authentication methods

- Additional features needed

3. Paste to any AI and get a complete Expo + Supabase project setup!

This prompt will generate everything you need: Expo project structure, Supabase database schema, authentication setup, navigation, screens, components, and configuration files - all ready to run! 🚀

AI Prompt: Generate SQL Script for Any Project

General Template for Database Schema Generation:

Create a complete pure SQL script file for a [PROJECT_TYPE] that matches all the tables I need for this project. Generate the database schema with the following requirements:

Project Requirements:

- Analyze the [PROJECT_TYPE] and create ALL necessary tables
- Include proper relationships between tables
- Add authentication tables if needed (user management)
- Include any junction tables for many-to-many relationships

Database Features:

- Enable Row Level Security (RLS) on all user-related tables
- Create policies for secure data access (users can only see their own data)
- Add CASCADE DELETE for proper cleanup when users are deleted
- Include indexes on frequently queried fields for performance
- Add triggers for automatic timestamp updates

Output Requirements:

- Create a folder called `superbase`
- Create a file called `database-schema.sql` inside the `superbase` folder
- The file must contain pure SQL script only (no JavaScript syntax)
- Include ALL SQL commands needed for the entire project (tables, indexes, policies, triggers)
- Format it so the entire file can be copied and pasted directly into Supabase SQL Editor
- Add SQL comments (using `--`) explaining each section
- Include setup instructions as SQL comments at the top of the file
- Ensure proper PostgreSQL syntax (parameters with defaults must be at the end)

The script should be production-ready with all necessary tables, constraints, indexes, and security policies for a multi-user [PROJECT_TYPE] application.

Expected File Format:

```
``sql
```

```
-- =====
```

```
-- Complete SQL Script for [PROJECT_TYPE]
```

```
-- =====
```

```
-- Setup Instructions:
```

```
-- 1. Copy this entire file content
```

```
-- 2. Go to Supabase SQL Editor
```

```
-- 3. Paste and run the script
```


-- 4. Enable Email authentication in Supabase Auth settings

-- [COMPLETE SQL SCRIPT GOES HERE]

-- All tables, indexes, policies, and triggers

-- Ready to copy/paste directly into Supabase

""

Comprehensive prompt template for design preferences in React Native/Expo projects:

Design Preferences: [Choose and customize the options below]

Design System:

Style Framework:

- NativeBase
- Tamagui
- UI Kitten
- Custom components
- Material Design
- Human Interface Guidelines

Design Language:

Modern

Minimalist

Colorful

Professional

Playful

Elegant

Component Library:

- Pre-built components
- Custom design system
- Hybrid approach

Color Scheme:

- **Primary Colors:** [Specific hex codes like #3B82F6, #10B981 or descriptions like "Blue and green", "Purple and orange"]
- **Theme Support:** [Light theme only/Dark theme only/Both light and dark themes]
- **Accent Colors:** [Colors for buttons, highlights, success/error states]
- **Background Style:** [Solid colors/Gradients/Patterns/Images]

Typography:

- **Font Family:** [System fonts/Custom fonts like Roboto, Inter, Poppins/Google Fonts]
- **Font Sizes:** [Small and compact/Medium and readable/Large and bold]
- **Text Hierarchy:** [Clear headings and body text/Minimal typography/Rich typography with multiple levels]

Layout & Spacing

- **Screen Layout:** [Card-based/List-based/Grid-based/Mixed layouts]
- **Spacing:** [Tight and compact/Comfortable padding/Spacious and airy]
- **Borders & Shadows:** [Flat design/Subtle shadows/Bold shadows/Rounded corners/Sharp corners]

Interactive Elements:

- **Buttons:** [Filled/Outlined/Text buttons/Floating action buttons/Custom button styles]

- **Form Elements:** [Material Design inputs/iOS-style inputs/Custom styled forms]
- **Navigation:** [Tab bar style, header design, gesture preferences]

Visual Elements:

- **Icons:** [Feather/Heroicons/Material Icons/Custom icon set/Specific style preference]
- **Images:** [Rounded/Square/Circular avatars/Image treatments]
- **Charts/Graphs:** [If needed - style preferences for data visualization]
- **Animations:** [Subtle animations/Rich animations/Minimal animations/Specific animation preferences]

Screen-Specific Design:

- **Authentication Screens:** [Clean and simple/Branded/Illustrated/Minimalist]
- **Main App Screens:** [Dashboard style/List views/Card layouts/Custom layouts]
- **Navigation Style:** [Bottom tabs/Drawer/Stack only/Custom navigation]

Brand Identity:

- **Logo/Branding:** [Include app logo/brand colors/specific brand guidelines]
- **App Icon Style:** [Flat/3D/Gradient/Minimalist/Detailed]
- **Splash Screen:** [Simple logo/Animated/Gradient background/Custom design]

Inspiration/References:

- **Similar Apps:** [Names of apps with designs you like]
- **Design Inspiration:** [Dribbble links/Pinterest boards/Specific design examples]
- **Must-Have Elements:** [Specific UI elements or patterns you want included]

Accessibility:

- **Text Size:** [Support for dynamic text sizing/Fixed text sizes]
- **Color Contrast:** [High contrast mode support/Standard contrast]
- **Touch Targets:** [Large touch areas/Standard sizing]

Example Usage:

Design Preferences:

Design System:

- Style Framework: NativeBase with custom theming
- Design Language: Modern and minimalist
- Component Library: Pre-built components with custom styling

Color Scheme:

- Primary Colors: #2563EB (blue) and #059669 (green)
- **Theme Support:** Both light and dark themes
- **Accent Colors:** #F59E0B (orange for highlights), #EF4444 (red for errors)
- **Background Style:** Solid colors with subtle gradients

Typography:

- Font Family: Inter (Google Font)
- Font Sizes: Medium and readable
- Text Hierarchy: Clear headings and body text with good contrast

Layout & Spacing:

- Screen Layout: Card-based with clean lists
- Spacing: Comfortable padding throughout
- Borders & Shadows: Subtle shadows with rounded corners

Interactive Elements:

- Buttons: Filled primary buttons with rounded corners
- Form Elements: Material Design inputs with floating labels
- Navigation: Bottom tab bar with icons and labels

Visual Elements:

- Icons: Feather icons for consistency
- Images: Rounded avatars and image cards
- Animations: Subtle animations for screen transitions
- Charts/Graphs: Clean, modern charts with brand colors

Screen-Specific Design:

- Authentication Screens: Clean and simple with centered content
- Main App Screens: Dashboard with cards and quick actions
- Navigation Style: Bottom tabs with 4-5 main sections

Brand Identity:

- Logo/Branding: Blue and green tech-focused branding
- App Icon Style: Flat design with gradient background
- Splash Screen: Simple logo with animated fade-in

Inspiration/References:

- Similar Apps: Notion, Todoist, Calm
- Design Inspiration: Clean SaaS interfaces, health app designs
- Must-Have Elements: Progress indicators, achievement badges

Accessibility:

- Text Size: Support for dynamic text sizing
- Color Contrast: High contrast mode support
- Touch Targets: Large, easy-to-tap buttons and interactive elements

This detailed design preference prompt will help the AI create a mobile app that matches your exact visual and UX preferences! 🎨

React Native with Expo: Essential Rules & Best Practices

🔴 **Critical Rules to Follow**

1. Expo SDK Limitations

- ✅ Use only Expo-compatible libraries - Check [Expo documentation] (<https://docs.expo.dev/>) before adding packages
- ❌ Cannot use native modules that require custom native code (unless using Expo Dev Client)
- ❌ No custom native code (Java/Swift/Objective-C) in managed workflow
- ✅ Use Expo modules for native functionality (camera, location, notifications, etc.)

2. Project Structure Rules

...

my-app/

├── app.json # Expo configuration (REQUIRED)

├── App.js/App.tsx # Main app entry point

├── assets/ # Images, fonts, etc.

├── components/ # Reusable UI components

├── screens/ # Screen components

├── navigation/ # Navigation setup

├── hooks/ # Custom React hooks

├── utils/ # Utility functions

├── constants/ # App constants

└── types/ # TypeScript types (if using TS)

3. Configuration Rules

```.json

// app.json - Essential configuration

{

  "expo": {

    "name": "Your App Name",

    "slug": "your-app-slug",

    "version": "1.0.0",

    "platforms": ["ios", "android"],

```
"orientation": "portrait",

"icon": "./assets/icon.png",

"splash": {

 "image": "./assets/splash.png"

},

"assetBundlePatterns": ["**/*"],

"ios": {

 "bundleIdentifier": "com.yourcompany.yourapp"

},

"android": {

 "package": "com.yourcompany.yourapp"

}

}

}

...

```



## Development Rules

### 4. Asset Management

-  Use `assets/` folder for images, fonts, icons
-  Use `require()` for static assets: `require('./assets/image.png')`
-  Optimize images before adding to project



- ❌ Don't use absolute paths for assets

## 5. Navigation Rules

- ✅ Use latest React Navigation (Expo-compatible)
- ✅ Install navigation dependencies: `expo install react-native-screens react-native-safe-area-context`
- ✅ Wrap app with `NavigationContainer`
- ❌ Don't use native navigation libraries

## 6. State Management

- ✅ Use React Context for simple state
- ✅ Use Redux Toolkit for complex state
- ✅ Use Zustand for lightweight state management
- ❌ Avoid prop drilling for shared state

## 7. Styling Rules

- ✅ Use `StyleSheet.create()` for performance
- ✅ Use Flexbox for layouts
- ✅ Use responsive design (`Dimensions`, `useWindowDimensions`)
- ❌ Don't use CSS files (not supported)

## Technical Rules

## 8. Dependencies

- ✅ Use `expo install` instead of `npm install` for Expo-compatible packages

-  Check package compatibility on [React Native Directory] (<https://reactnative.directory/>)
-  Keep Expo SDK updated to latest stable version
-  Don't install packages that require native linking

## 9. Performance Rules

-  Use FlatList for large lists (not ScrollView)
-  Implement lazy loading for images
-  Use React.memo for expensive components
-  Optimize images (WebP format when possible)
-  Don't render large lists with map()

## 10. TypeScript Rules (if using)

-  Use TypeScript for better development experience
-  Type all props and state
-  Use interfaces for data structures
-  Enable strict mode in tsconfig.json

## Security Rules

### 11. Environment Variables

-  Use `expo-constants` for environment variables
-  Keep sensitive data in secure storage
-  Don't put secrets in source code
-  Don't commit .env files with secrets

## 12. Data Protection

-  Use `expo-secure-store` for sensitive data
-  Validate all inputs from users
-  Use HTTPS for all network requests
-  Don't store passwords in plain text

## Build & Deployment Rules

### 13. Build Configuration

-  Use EAS Build for production builds
-  Configure app signing properly
-  Test on physical devices before release
-  Don't use Expo CLI for production builds (deprecated)

### 14. App Store Rules

-  Follow platform guidelines (iOS Human Interface, Android Material Design)
-  Provide required assets (icons, splash screens, screenshots)
-  Test on different screen sizes
-  Handle permissions properly

### 15. Version Management

-  Update version in app.json before builds
-  Use semantic versioning (major.minor.patch)
-  Keep changelog of changes

-  Test updates thoroughly

## Performance Best Practices

### 16. Memory Management

-  Clean up subscriptions in useEffect cleanup
-  Remove event listeners when components unmount
-  Use PureComponent or React.memo for static content
-  Don't create objects inside render methods

### 17. Network Rules

-  Handle offline scenarios gracefully
-  Implement loading states for all network requests
-  Cache data when appropriate
-  Use error boundaries for error handling

### 18. Testing Rules

-  Write unit tests for business logic
-  Test on both iOS and Android
-  Use Expo Go for development testing
-  Test edge cases (no internet, low battery, etc.)

## Common Pitfalls to Avoid

### 19. What NOT to do:

-  Don't use libraries that require native code
-  Don't ignore platform-specific differences

- ❌ Don't hardcode device dimensions
- ❌ Don't use index as key in FlatList
- ❌ Don't mutate state directly
- ❌ Don't ignore memory leaks
- ❌ Don't skip error handling

## 20. Development Workflow

- ✅ Use Expo Dev Client for custom native code (if needed)
- ✅ Use EAS Update for over-the-air updates
- ✅ Use Expo Application Services for CI/CD
- ✅ Test on Expo Go during development

## Pre-Launch Checklist

- [ ] App works on both iOS and Android
- [ ] All assets are optimized
- [ ] App.json is properly configured
- [ ] Permissions are handled correctly
- [ ] Network requests have error handling
- [ ] Loading states are implemented
- [ ] App is tested on physical devices
- [ ] App store assets are ready
- [ ] Version numbers are updated

- [ ] Security best practices are followed

Following these rules will help you build robust, performant, and maintainable React Native apps with Expo! 🚀

# BUILD MOBILE APPS WITH NO CODE



This Docs offers a comprehensive prompt template for building React Native mobile applications using Expo and Supabase. This guide streamlines the project setup, covering everything from app structure and authentication to database schema and UI design preferences. With detailed instructions and best practices, it empowers developers to create scalable, production-ready apps efficiently.